Lab2 BombLab 实验报告

实验目标与分析方法

根据个人炸弹的汇编恢复计算规则，再把规则写进 `defuser.c`。 同一关的 challenge 会随 seed 改变，因此一次手动通关只是验证某组输入，求解函数还需要适用于其他 challenge。

我在 Ubuntu WSL 的 Linux x86-64 环境下完成实验，使用 GCC、Make、objdump 和 GDB，只修改了 `defuser.c`。本文对应 bomb 的 SHA-256 为 `9eb6500d04b398ab127e297f4f1e040f132881421351c57e0eb7ab81e0452175`。

一开始，虽然我能够按命令查看寄存器，却没能马上理解这些数与最终解法有什么关系。后来我通过把整个过程分成读出检查条件、恢复计算关系、观察运行值、编写通用函数四步总结出来了四个流程。

- 1.看汇编提供的规则
- 2.由 GDB 检查我对规则的理解是否正确，
-
    3. C 程序自动生成口令。

第 1 关：函数调用与参数传递

我首先确认了口令格式，再沿函数调用追踪 challenge 的去向。入口函数把计算交给了 `p1_compute`，后者调用辅助函数。
`phase_01` 检查一个整数。 `mov %rsi,%r12` 保存 challenge， `call p1_compute` 返回期望值，再用 `cmp %eax,-0x14(%rbp)` 比较输入。
我按照实际指令追踪，得到 step1、step0、step2；结合每次调用前的参数准备和辅助函数实际改写的寄存器，我整理出了：

```
high = U32(C >> 32)
low = U32(C)
shifted = U32(C >> 16)
mid1 = U32(high * 5) XOR U32(shifted + 0x6b8b)
mid0 = (high XOR 0x5d40) + U32(low * 4)
answer = U32(mid0 * 3 + mid1) XOR 0xe025
```

关键依据如下：

- `lea (%rdi,%rdi,4),%eax` （乘 5）
- `xor $0x5d40,%esi` 、 `lea (%rsi,%rdi,4),%eax` ，
- `lea (%rdi,%rdi,2),%eax` 、 `add %esi,%eax` 、 `xor $0xe025,%eax` 。
  GDB 对 seed 42 核对，challenge 的值为 `0x28efe333b266f103` ，三个计算返回值依次为 798716430、4069229183、121444782，与恢复公式一致。
  输入 121444782 后出现 `Phase 1 defused.` 。

第 1 关让我认识到：

- 寄存器并不固定代表某个变量。一次调用结束后，必须结合指令重新判断其含义。
- `lea` 不一定是在访问内存，例如 `lea (%rdi,%rdi,4),%eax` 在这里就是乘 5。把它翻译成算术表达式后，原来零散的指令才能连接成计算过程。
  额外 seed：

```
whalerealnot@localhost:~/ICS_jc26fall/ics-2026-bomblab$ { ./defuser --seed 314159; ./defuser --secret --seed 314159; } | ./bom
seed 314159

Phase 1 challenge: 0xd85dda4696932a2d
Input (unsigned decimal integers):
Phase 1 defused.
```

第 2 关：数组、循环与递推

`sscanf` 要求成功读取六个整数。数组起点是 `rbp-0x40`，`mov -0x40(%rbp,%rdx,4),%esi` 通过“起点 + 下标×4”读取元素。
初始值由 `shr $0x2d,%rax; and $0xf,%eax; xor $0xd,%eax` 得到。循环从下标 1 开始，`ecx` 初值 22，每轮加 3；`rdx` 每轮加 1，达到 6 时结束。

```
v[0] = ((C >> 45) AND 15) XOR 13
对于 i = 1..5:
    shift = 22 + 3*(i-1)
    part = (C >> shift) AND 31
    v[i] = U32(part + i) XOR U32(v[i-1]*3)
```

在 seed 42、challenge `0x47526757130f9f52` 下观察到首值 14，首轮下标：1、移位：22、下一值：39，均与公式相吻合。

调试这关时出现了小插曲，我在输入六个占位值后直接继续运行，程序并没有在预期比较位置停下，而是爆炸退出，再查看寄存器提示 `No registers`。询问ai后我得知这不是公式的错误，而是进程已经结束了。我重新设置比较位置的临时断点，确认停在 `0x40216a` 后再次核对期望值与占位输入。

确认第一轮后进行推广，下标从 1 到 5，移位从 22 开始每轮加 3，对应 `22 + 3*(i-1)`。下面是额外 seed 314159 的实际运行截图。

```
whalerealnot@localhost:~/ICS_jc26fall/ics-2026-bomblab$ { ./defuser --seed 314159; ./defuser --secret --seed 314159; } | ./bomb --
seed 314159
Phase 2 challenge: 0x049392bee0cdfbbd
Input (unsigned decimal integers):
Phase 2 defused.
```

第 3 关：switch 与跳转表

这一关需要解决选择编号-跳转目标-实际计算的对应关系，选择编号来自跳转表的位置，和函数名称无关。
首先要读取两个整数，第一个数为 `index = C AND 7`，第二个数等于分派计算结果，公共参数是 `x = U32(C >> 32) XOR U32(C >> 3)`。
`notrack jmp *0x405820(,%rax,8)` 从每项 8 字节的表读取代码地址。通过用 GDB 的 `x/8gx 0x405820` 读取八个表项，我得到了如下数据：

编号	表中目标	实际函数	32位计算
0	0x402288	p3_case_2	x*5 + 0x49c9
1	0x402290	p3_case_7	x*5 + 0xf483
2	0x402298	p3_case_1	(x XOR 0x9f5d) + (x >> 2)
3	0x4022a0	p3_case_0	x*5 + 0x4b5b4
4	0x4022a8	p3_case_3	(x*4) XOR 0x624
5	0x4022b0	p3_case_6	x*3 + 0x6047
6	0x402278	p3_case_5	(x XOR 0xb16a) + (x >> 1)
7	0x402280	p3_case_4	(x*4) XOR 0x6fc

seed 42、challenge `0x581ce1ff0e4ae394` 下编号为 4，进入 p3_case_3。参数 3117792653，返回值为 3881234448，输入 `4 3881234448` 后通过，编号 4 确实是 p3_case_3，表达式与实际返回值也相同。
有一部分分支先把 x 复制到 `eax`，再右移 `edi`，这代表计算中同时使用了原值和移位后的值。

```
whalerealnot@localhost:~/ICS_jc26fall/ics-2026-bomblab$ { ./defuser --seed 314159; ./defuser --secret --seed 314159; } | ./bomb --
seed 314159
Phase 3 challenge: 0x1cd4d8174d7decfd
Input (unsigned decimal integers):
Phase 3 defused.
```

第 4 关：递归调用与返回

这一关需要理解向下调用和向上返回。读取一个整数。调用参数为：

```
target = C AND 127
low = 0, high = 127
key = U32(C >> 7) XOR U32(C >> 32)
```

`sub %esi,%edi; shr $1,%edi; add %esi,%edi` 计算中点 `mid = low + ((high-low)>>1)`。

- `je`： 进入终止分支
- `jb`： 按无符号比较决定左半区
- 递归保留 target 和 key，修改区间边界

```
找到目标：返回 (mid + 0xed) XOR key
目标在左：child = walk(target,low,mid-1,key)
          返回 U32(child*3) XOR U32(mid+key) XOR 0x17
目标在右：child = walk(target,mid+1,high,key)
          返回 U32(child*3) XOR U32(mid+key) XOR 4
```

seed 42 下观察到 target=114，key=3186935837，区间依次为 `[0,127]`、`[64,127]`、`[96,127]`、`[112,127]`、`[112,118]`、`[112,114]`、`[114,114]`，最深层返回 3186936130，上一级加工后为 2217382732，最终口令 1438677014 正常通过。

我先在入口观察两层参数，第一次区间为 `[0,127]`，中点 63，小于目标 114。下一层区间变成 `[64,127]`，目标和 key 没有发生变化。`bt` 中同时出现了两层 `p4_walk`，这表示上一层还在等待子调用。

之后我在终止分支设断点，再用 `finish` 返回一层，分别验证最深层返回值和父层的加工结果。一次 `finish` 只结束当前这一层，返回到的 `eax` 有可能会继续被上层修改。因此我最后停到了 `phase_04` 的比较指令处后才把 `eax` 当作最终口令。

```
whalerealnot@localhost:~/ICS_jc26fall/ics-2026-bomblab$ { ./defuser --seed 314159; ./defuser --secret --seed 314159; } | ./bomb --
seed 314159

Phase 4 challenge: 0x6a9e8d40d5354fbc
Input (unsigned decimal integers):
Phase 4 defused.
```

上图记录了额外 challenge `0x6a9e8d40d5354fbc` 的通过结果。

第 5 关：结构体布局与链表

`p5_traverse` 通过 `eax` 返回第一个结果，通过第二个指针参数写出另一个结果，再分别与两个输入比较。这样可以先确定输出顺序，再处理链表计算。

关键内存访问：

```
mov 0x405b60(,%rax,8),%rdx  # 从起点指针表选节点
mov 0x8(%rdx),%esi          # 节点 +8：32位编号
add 0xc(%rdx),%eax          # 节点 +12：32位权重
mov (%rdx),%rdx             # 节点 +0：8字节 next 指针
mov %esi,(%r8)             # 写回最近访问节点的编号
```

在整理节点布局时按照偏移和访问宽度区分字段。next(+0,8字节)、id(+8,4字节)、weight(+12,4字节)。一共 12 个静态节点，从 `0x405ba0` 起隔 16 字节排列。
起点表对应节点下标为 `[6,1,4,7,2,3,0,5]`，节点表如下：

下标	next下标	id	weight	
0	5	0x32	0x43d6	
1	4	0x25d	0x9ef9	
2	3	0x652	0x36eb	
3	0	0x442	0x10cc	
4	7	0x50b	0xfe10	
5	8	0x17b	0xd964	
6	1	0xb4f	0x5ee7	
7	2	0x929	0x5b7b	

下标	next下标	id	weight	
8	11	0x728	0x47bf	
9	10	0x81c	0xbcc7	
10	空	0x357	0x88b8	
11	9	0xa57	0xbd00	

`visits = 1 + ((C>>3) AND 3)`，访问 1～4 个节点。程序先处理首节点，再决定是否沿 next 前进。

```
node = heads[C AND 7]
acc = U32(C>>6) XOR U32(C>>32) XOR 0xd4b4
每个访问节点：
    acc = U32((acc + weight)*5) XOR id
    last_id = id
    node = next
输出 acc、last_id
```

第 6 关：函数指针表与间接调用

这一关与第 3 关都需要读取地址表，但表项的用途不同。第 3 关是通过表跳转到分支，这一关则使用 `call *0x405e00(,%rax,8)` 和 `call *0x405e00(%rbx)` 连续调用两个函数，返回后继续计算。为避免混淆，把两次选择的编号分别记作 i、j。
从原二进制读取的表按编号对应 `op3, op0, op4, op1, op2, op1, op5, op5`，含重复项。

```
x = U32(C>>6) XOR U32(C>>32)
i = C AND 7
j = (C>>3) AND 7
t = op[i](x) XOR 0x47fb
answer = U32(op[j](t) + 0x13cd)
```

六个函数的规则：

函数	32位计算
op0	ROR32(x,1)
op1	((x>>5) XOR x) + 0xfbb5
op2	(x*8 + 0x4b81) XOR (x*3)
op3	(x XOR 0x4f79) + (x<<5)
op4	ROL32(x,5) XOR 0x9532
op5	(x + 0xc5d9) XOR (x>>1)

`ror`、`rol` 不能直接替换为普通 `>>`、`<<`。例如右循环移位 1 位，需要把原最低位补到最高位，`(x >> 1) | (x << 31)`。两次函数选择还必须按正确顺序执行，先异或 `0x47fb` 再加 `0x13cd`。

Honor：入口布局与上下文校验

这部分如果只看 `secret_phase` 的三个整数公式，就会漏掉外层 `secret_gate` 的格式要求。外层先复制整行，检查偏移 32 处的 4 字节标记，再把偏移 36 处的文本交给 `secret_phase`。

```
lea -0xd0(%rbp),%r12
call secret_copy
cmpl $0x4e45504f,-0xb0(%rbp)
lea -0xac(%rbp),%rdi
call secret_phase
```

偏移差分别为 `0x20=32` 和 `0x24=36`。小端序下 `0x4e45504f` 对应 ASCII `OPEN`。入口要求行长 36～163 字节。输出构成为 32 个 A、OPEN、三个以空格分隔的整数。
令 `T=context`，恢复数值校验：

```
selector = U32(T) AND 3
fc = U32(C) XOR U32(C>>32)
ft = U32(T) XOR U32(T>>32)
mixed = U32((fc XOR ft)*7 + 0xefed) XOR 0x8ad1
first = mixed XOR U32(selector*0xefed) XOR U32(T>>16)
second = first XOR U32(ft + selector + 0x8ad1 + (fc>>5))
```

第七行数值区依次写 selector、first、second。context 由公共驱动根据前六行口令计算，求解函数只使用传入值，不需要读文件、创建进程或访问网络。
验证 Honor 时还必须考虑 context 的来源。它由前六行口令计算得到，因此第七行不能脱离前六关单独验证。

公开列表之外的输入

seed 314159 不在公开列表中。下表是完整求解驱动的输出，并通过了个人 bomb 的辅助指令核验。

关卡	seed 314159 的求解输出
1	2073047115
2	1 7 7 4 45 147
3	5 3765604575
4	2496674862
5	2075544647 1618

关卡	seed 314159 的求解输出
6	1776374798
Honor	32个A + OPEN0 1009258468 553349767

测试记录

```
whalerealnot@localhost:~/ICS_jc26fall/ics-2026-bomblab$ make test
Phase 6 seed 2026: PASS
Phase 6 seed 65535: PASS
Phase 6 seed 987654321: PASS
Phase 6 seed 18446744073709551615: PASS
Public tests: 60/60
Secret seed 0: PASS
Secret seed 1: PASS
Secret seed 7: PASS
Secret seed 42: PASS
Secret seed 99: PASS
Secret seed 123: PASS
Secret seed 2026: PASS
Secret seed 65535: PASS
Secret seed 987654321: PASS
Secret seed 18446744073709551615: PASS
Secret public tests: 10/10
```

实验体会

`defused` 并不代表着题目被解决了。公开测试、额外输入和中间结果核对提供的是不同层面的验证。

通过这次实验，我对循环、递归、指针都有了更加深刻的了解。最初我直接跟着指令和寄存器走，只看到了当前数值，却不清楚它代表的是什么。后来我改变了解题顺序，先确认每关要检查什么，再给参数和中间值起名，最后回到 GDB 核对，程序的数据流才更加清晰。

几种控制方式虽不相同，但也有共同点。循环保留了上一轮的状态，递归保留了每层状态，跳转表和函数指针表决定了下一段代码从哪里开始执行。我发现通过把这些变化写成伪代码，求解还原的过程变得更加顺畅。